

—— 大数据工具课 · L5-L6

Spark 实战 · 第 5-6 课

Spark SQL 实战 × 性能调优

Spark SQL in Practice · Performance Tuning

本系列讲什么

What We Cover

L5

05

Spark SQL 实战

Spark SQL in Practice

01 DataFrame API vs SQL

02 5 大核心算子

03 电商实战迁移

04 用户行为漏斗

bigdata-tools · 课程地图

L6

06

性能调优

Performance Tuning

01 数据倾斜原理与定位

02 加盐打散

03 广播 Join

04 调优参数与 AQE



05

Spark SQL 实战

Spark SQL in Practice

会 SQL，就会 Spark SQL —— 同一套 **Catalyst** 引擎，**DataFrame** 与 **SQL** 殊途同归。

5.1

DataFrame 与 SQL 入口

5.2

读写与 Schema

5.3

转换 · 聚合 · Join

5.4

Catalyst 执行计划

L5 DataFrame API vs Spark SQL Two Ways, One Engine

同一逻辑，两种写法——经 **Catalyst** 优化后性能完全相同。

A DataFrame API

```
result = df \
    .filter(col("status") ≠ "cancelled") \
    .join(products, on="product_id", how="left") \
    .groupBy("category") \
    .agg(count("order_id").alias("cnt"),
         sum("amount").alias("gmV")) \
    .orderBy(col("gmV").desc())
```

B Spark SQL

```
spark.sql("""
    SELECT category,
           COUNT(order_id) AS cnt,
           SUM(amount)     AS gmV
    FROM orders o
    LEFT JOIN products p ON o.product_id = p.product_id
    WHERE status ≠ 'cancelled'
    GROUP BY category
    ORDER BY gmV DESC
""")
```

Catalyst → 逻辑计划 → 物理计划 → RDD

两种写法 **殊途同归**，优化器最终生成相同的执行计划。

用 **API** 适合动态构建查询、**Python** 团队工程化。

用 **SQL** 适合 **Hive** 迁移、与 **BI** 工具对接。

L5 五大核心算子 Core Operators

这 5 个动作，覆盖 90% 的离线数据处理任务。

01 DataFrame 基础 basics

读数据 · `printSchema` · `show` · `filter` 行过滤 · `select` 列选择

02 GroupBy 聚合 groupBy

相同 **key** 归组做统计 —— 触发 **Shuffle**（性能瓶颈）

03 Join join

inner / left / right / full outer 四型 —— 小表 `broadcast` 避免 Shuffle

04 窗口函数 window

分组后逐行计算，**保留所有行**（vs GroupBy 压缩成一行）

05 Spark SQL spark.sql

`createOrReplaceTempView` 注册视图，直接写 SQL

L5 GroupBy 与 Shuffle The Cost of Regrouping

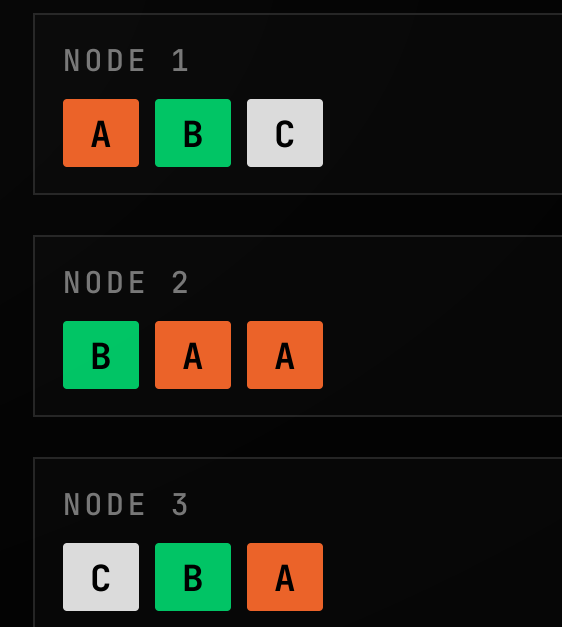
把相同 key 的行归到一组做统计 —— 底层触发 **Shuffle**，数据在节点间重新分布。

GROUPBY.PY

```
result = df.groupBy("user_id") \
    .agg(count("order_id").alias("trade_count"),
         sum("amount").alias("total_amount"),
         round(avg("price"), 2).alias("avg_price")) \
    .orderBy(col("total_amount").desc())
```

SHUFFLE

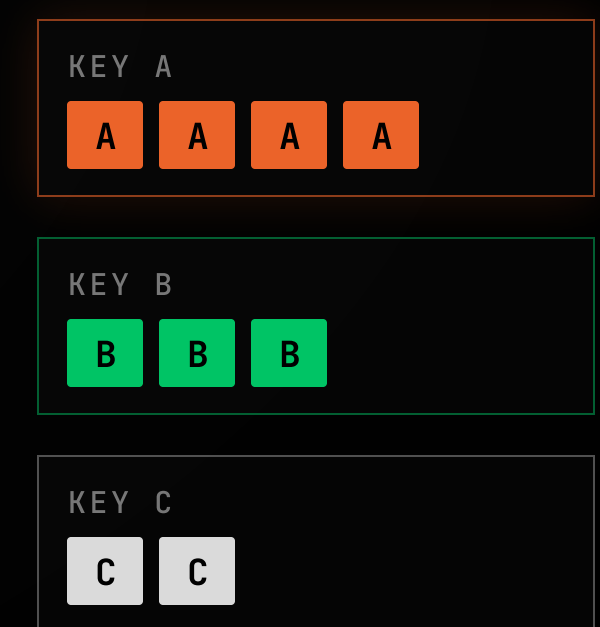
BEFORE · 数据分散



→
SHUFFLE
网络重分布

KEY → NODE

AFTER · 按 KEY 汇聚



- › `groupBy().agg()` 是标准聚合写法
- › GroupBy 触发 **Shuffle**，是性能瓶颈所在 — 为 L6 数据倾斜埋伏笔

L5 Join 操作

Four Types + Broadcast

两张表按共同 key 拼接 —— Join 也会触发 Shuffle，除非广播小表。

01 / INNER

inner



只保留两边都有的

02 / LEFT

left



保留左表所有行，右表缺则 null

03 / RIGHT

right



保留右表所有行

04 / FULL

full outer



两边都保留

BROADCAST_JOIN.PY

```
from pyspark.sql.functions import broadcast
# orders (100亿行) JOIN users (5000行) — 小表直接广播
fast = orders.join(broadcast(users), on="user_id", how="left")
# 自动广播阈值默认 10MB
spark.conf.set("spark.sql.autoBroadcastJoinThreshold", "50m")
```

BROADCAST 小表 broadcast 到每个节点 → 大表零 Shuffle，性能提升巨大。

L5

窗口函数

Window Functions

分组后对每行单独计算 —— 与 GroupBy 的本质区别：窗口函数保留所有行。

GROUPBY().AGG()

GroupBy

N 行 → 1 行 / 组 (压缩)

• user_1 · 80

• user_1 · 120

• user_1 · 50

聚合压缩 ↓

• user_1 · sum = 250

3 行变 1 行，
原始明细被吃掉

.OVER(WINDOW)

窗口函数

N 行 → N 行 + 新列 (不减行)

+ RANK_IN_USER

• user_1 · 120

rank 1

• user_1 · 80

rank 2

• user_1 · 50

rank 3

逐行计算 ↓

• 3 行原样保留 + 1 新列

行数不变，
每行都带上组内信息

WINDOW.PY

from pyspark.sql import Window

L5 窗口函数from pyspark.sql.functions import rank, sum, lag

- KEY POINTS
- ▶ partitionBy 定义分组，orderBy 定义排序

▶ 常用 rank / lag / lead / rowsBetween

▶ 适合 排名、累计、移动平均


```
w = Window.partitionBy("user_id").orderBy(col("amount").desc())
```

```
df.withColumn("rank_in_user", rank().over(w))      # 组内排名
```

```
df.withColumn("cum", sum("amount").over(w))        # 累计
```

```
df.withColumn("prev", lag("price", 1).over(w))     # 上一行值
```

L5 电商实战迁移 Hive → Spark SQL

典型 Hive SQL → Spark SQL 迁移 —— 三个常见查询场景。

• gmV.py

```
df = spark.read.parquet("oss://bucket/orders/")
df.createOrReplaceTempView("orders")

spark.sql("""
    SELECT date_format(create_time, 'yyyy-MM') AS mon,
           category, SUM(amount) AS gmV,
           COUNT(DISTINCT user_id) AS buyers
    FROM orders
    WHERE create_time ≥ '2024-01-01' AND status = 'paid'
    GROUP BY 1, 2 ORDER BY mon, gmV DESC
""").show()
```

迁移要点

01 **DATE_FORMAT** 等函数同名直用, SQL 几乎零改动

02 表路径从 **Hive Metastore** 改为直读 **Parquet**

L5 • 电商实战 • Hive → Spark SQL
03 **create_time** 过滤自动触发 **partition pruning**

三个查询场景

01 GMV 统计

按月 + 品类 **SUM(amount)**, COUNT(DISTINCT user_id) 统计
买家数

02 复购率

30 天内下过 **≥2** 单的用户比例 ——
WITH user_orders ... CASE WHEN order_cnt ≥ 2

03 销售排名

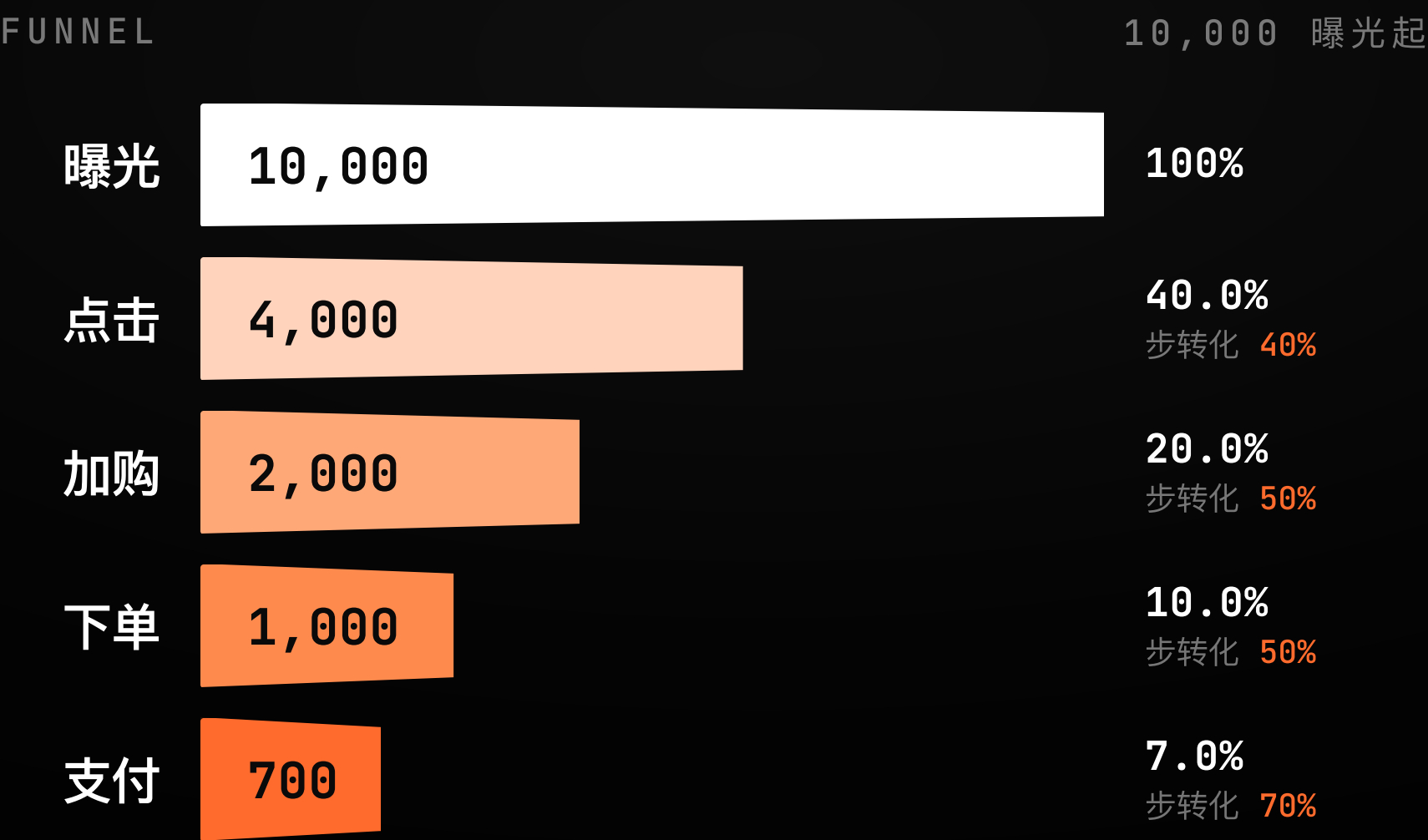
每品类 **TOP3** ——
RANK() OVER (PARTITION BY category ORDER BY sales
DESC), WHERE rnK ≤ 3

L5

用户行为漏斗

Conversion Funnel

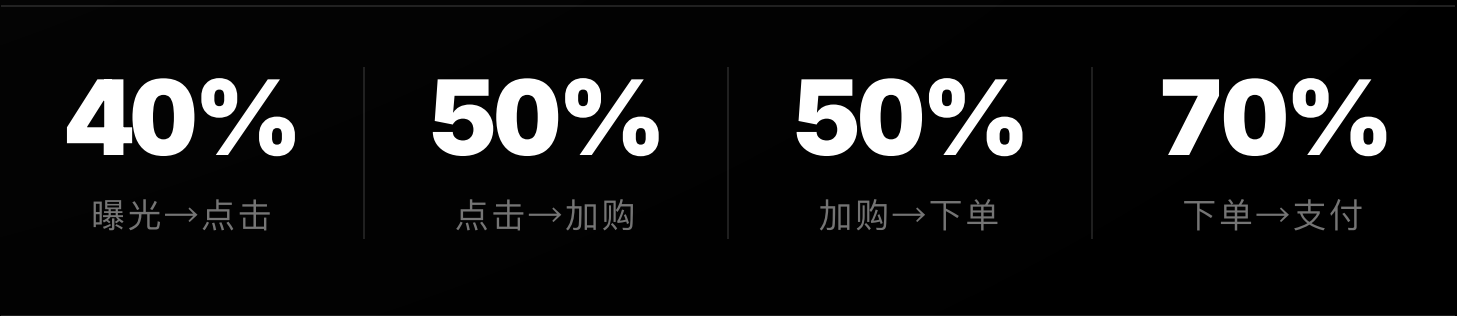
曝光 → 点击 → 加购 → 下单 → 支付，**五步转化**。



整体转化率

7%

10,000 曝光 → 700 支付



MAX(CASE WHEN event_type='click' THEN 1 ELSE 0 END) AS
step_flags // 每个 user 聚出各步 flag，再逐层相乘得分层人数

SECTION · L6



性能调优

Performance Tuning

当 **99** 个 **Task** 都完成、第 **100** 个还在跑 —— 是时候谈调优了。

数据倾斜 / 定位 / 加盐 / 广播 Join / 调优参数 / AQE

L6

数据倾斜

Data Skew

为什么 99 个 Task 都完成了，第 100 个还在跑？

99 TASKS

2

s

// 早已空闲

- VS -

1 TASK

47

min

// 一个 Task 扛了 90% 的数据

TASK DURATION



SPARK UI · STAGES

成因 / 典型场景

GroupBy / Join 后，某个 key 的数据量远大于其他 key，一个 Task 处理了 90% 数据，其余早已空闲。电商日志里"爬虫账号"产生百万条记录；Join 时 NULL 值被当成

两种武器

- 1 加盐打散 GroupBy 倾斜
- 2 广播 Join Join 倾斜

同一个 key 聚合。

L6 倾斜定位 Locate the Skew

Spark UI → Stages → 找 **max duration** 远大于 **median** 的 Stage。

1 Spark UI → Jobs → Stages → Tasks, 看 **max vs median**

// 单个 Stage 内任务耗时分布

2 看 Task 的 **Shuffle Read Size** 分布是否不均

// 数据量集中 = 倾斜信号

3 检查 **NULL key**

// NULL 在 JOIN 时全聚到一个 Task

■ find_skew.py

```
# Step 1: 找出哪个 key 数据量异常
df.groupBy("user_id").count() \
    .orderBy(col("count").desc()).show(20)

# Step 2: 统计 key 分布
df.groupBy("user_id").count().selectExpr(
    "percentile_approx(count, 0.5) as median",
    "percentile_approx(count, 0.99) as p99",
    "max(count) as max_count").show()

# Step 3: NULL key 检查
df.filter(col("user_id").isNull()).count()
```

L6 加盐打散 Salting

给倾斜 key 加随机后缀**打散成 N 份** —— 局部聚合后再去盐合并。

01 · 问题

倾斜 key

user_id=BOT
独占 90% 数据



02 · 打盐

加随机盐

BOT_0 ~ BOT_9
裂成 10 份



03 · 局部聚合

分散到 10 Task

无倾斜 · 各算各的
partial_sum



04 · 去盐

全局再聚合

split 去后缀
合并 → total

>_ salt_groupby.py

```
SALT_NUM = 10
# Step1 打盐
salted = df.withColumn("salted_key",
    concat_ws("_", col("user_id"), floor(rand()*SALT_NUM).cast("string")))
# Step2 局部聚合 (key 已分散, 无倾斜)
partial = salted.groupBy("salted_key").agg(sum("amount").alias("partial_sum"))
# Step3 去盐 + Step4 全局再聚合
partial = partial.withColumn("user_id", split(col("salted_key"), "_")[0])
result = partial.groupBy("user_id").agg(sum("partial_sum").alias("total_amount"))
```


L6 广播 Join Broadcast Join

小表广播到每个节点 —— 大表零 **Shuffle** 本地 join。

1 大表 Join 小表

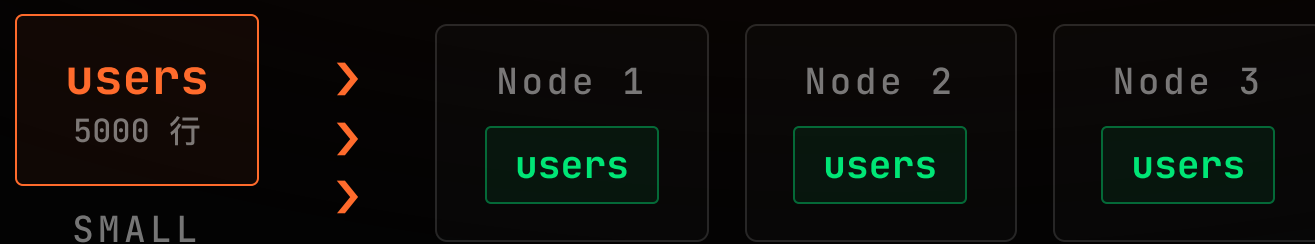
默认走 Shuffle — 大表全网搬运

2 broadcast(小表)

小表完整副本发到 每个节点

3 各节点本地 Join

零 **Shuffle** · 大表数据原地不动



■ broadcast_join.py

```
from pyspark.sql.functions import broadcast
# orders (100亿行) JOIN users (5000行)
result = orders.join(broadcast(users), on="user_id", how="left")

# 两张大表都有倾斜 key? 先分离倾斜部分分别处理
normal = orders.filter(~col("user_id").isin(SKEW_KEYS))
skewed = orders.filter( col("user_id").isin(SKEW_KEYS))
final = normal.join(users, "user_id", "left") \
    .union(skewed.join(broadcast(users), "user_id", "left"))
```

L6

调优参数速查

Tuning Cheatsheet

面试必考 · 生产必调 —— 最常调的是 **spark.sql.shuffle.partitions**。

PARAMETER	DEFAULT	说明 / 建议
spark.sql.shuffle.partitions 最常调	200	Shuffle 后分区数。数据量大调大(1000)，小调小(50)。公式：数据量(GB) ×2~4
spark.executor.memory	1g	每 Executor 内存。 OOM 调大，单个不超 64g。生产 4g~16g
spark.executor.cores	1	每 Executor CPU 核。推荐 2~5。Task 并发 = cores × instances
spark.sql.adaptive.enabled	true 3.x	AQE 自适应执行，运行时动态调分区 + 处理倾斜， 强烈保持开
spark.memory.fraction	0.6	Spark 内存池占堆比例。 OOM 可调到 0.75
spark.sql.autoBroadcastJoinThreshold	10MB	小于此值的表 自动广播 。推荐 20m~100m

L6 AQE 自适应查询执行 Adaptive Query Execution

Spark 3.0+ **默认开启** —— 运行时动态调整，省去大量手动调参。

• SPARK_CONFIG.PY

```
spark = SparkSession.builder.appName("prod-job") \
    .config("spark.executor.memory", "8g") \
    .config("spark.executor.cores", "4") \
    .config("spark.sql.shuffle.partitions", "200") \
    .config("spark.dynamicAllocation.enabled", "true") \
    # —— AQE: 强烈推荐开启 ——
    .config("spark.sql.adaptive.enabled", "true") \
    .config("spark.sql.adaptive.skewJoin.enabled", "true") \
    .getOrCreate()
```

AQE 三大能力

1 运行时**动态调整** shuffle 分区数

按真实数据量重算，不再死守 200

2 自动检测并处理**数据倾斜**

skewJoin · 拆分超大分区

3 动态**合并小分区**

省去手动调参

TAKEAWAY 开启 **AQE**，让 **Spark** 自己调优。

L5-L6 两课回顾 & 作业

Recap & Homework

从 **Spark SQL 实战** 到 **性能调优** —— 把三件事刻进肌肉记忆。

—— 记住三件事

1 DataFrame 与 SQL 殊途同归

都走 **Catalyst** 优化器 —— 同一执行计划，性能完全相同。

2 Shuffle 是性能瓶颈

GroupBy / Join 触发 —— 数据倾斜由它而来。

3 调优先开 AQE

再谈 **加盐 / 广播 / shuffle.partitions**。

—— 两课作业

HW5 用户行为漏斗分析

≈ 2-3h

Spark SQL 版：生成模拟日志 → 跑漏斗 **SQL** → 输出**转化率**

HW6 数据倾斜 复现→定位→修复

≈ 3-4h

造 **100 万行** (BOT001 占 80%) → Spark UI 截图对比加盐前后耗时
→ 开 **AQE** 验证

读懂计划，驯服 **Shuffle**。

— bigdata-tools